



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

A **State Space** represents all possible states that an agent can reach in a problem environment, together with the actions or transitions between those states. For example, in the grid game, each valid grid position can be considered a state, while moving Up, Down, Left, or Right represents an action that changes the state.

A **Search Tree**, on the other hand, is a structure generated by a search algorithm from the initial state. It represents the different paths that can be explored to reach a goal state. The same state can appear more than once in a search tree if it is reached through different paths.

Therefore, the main difference is that the **state space represents the possible states of the problem**, while the **search tree represents the paths explored by the search algorithm through those states**.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The `reached` set is used to keep track of states that have already been visited by the search algorithm. It prevents the algorithm from expanding the same state repeatedly.

This is particularly important in a grid because the agent can move between neighbouring cells and return to a previously visited cell. Without a `reached` set, the search algorithm could repeatedly explore the

same states and create cycles.

In the case of **DFS**, the absence of a `reached` set can cause the algorithm to continuously follow cycles in the search space. For example, the agent could move from one cell to another and then return to the first cell repeatedly. As a result, DFS may continue indefinitely or perform a very large amount of unnecessary searching.

Therefore, the `reached` set improves the efficiency of graph search and prevents repeated exploration of already visited states.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

Breadth-First Search (BFS) explores the search space level by level. It first examines all states that are one step away from the starting state, then all states that are two steps away, and so on. Therefore, when every movement has the same cost, BFS finds the shortest path to the goal.

Depth-First Search (DFS) follows one path as deeply as possible before backtracking and exploring another path. Because of this strategy, DFS can enter a long or winding route even when a shorter route exists elsewhere in the grid.

In this grid environment, each movement has an equal cost. Therefore, BFS is able to find a path with the minimum number of movements. DFS does not guarantee the shortest path because it prioritizes depth rather than path length.

Thus, BFS generally produces a shorter path in this unit-cost grid, while DFS can produce a longer and more winding path depending on the order in which neighbouring states are explored.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive

1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

The main difference between BFS and DFS on a very large grid is their memory usage.

For BFS, the approximate time and space complexities are:

- **Time:** $O(b^d)$
- **Space:** $O(b^d)$

where **b** is the branching factor and **d** is the depth of the shallowest solution.

BFS stores a large frontier of unexplored nodes. As the search depth increases, the number of nodes stored in memory can grow exponentially. Therefore, on a 1000×1000 grid, the major bottleneck for BFS is **memory usage**, especially when the solution is relatively deep.

For DFS, the approximate complexities are:

- **Time:** $O(b^m)$
- **Space:** $O(bm)$

where **m** is the maximum depth of the search tree.

DFS stores mainly the current path and the remaining branches at each level, so its memory requirement is much lower than BFS. However, DFS may spend a long time exploring an unsuitable deep path before finding the goal.

Therefore, on a 1000×1000 grid, **BFS can face a significant memory bottleneck**, while **DFS uses considerably less memory but may require more searching time and does not guarantee the shortest path**.

Once Completed Get a PDF of the completed File and Push into the Week

03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING